INDEX

II yr / III sem

NAME : SATHYA NARAYANAN. R STD : ..... SEC : ..... ROLL NO : 2510021240

SUBJECT : DAA LAB (OBSERVATION) SCHOOL : .....

S.No.	Date	Title	Page No.	Teacher's Sign / Remarks
		I - Finding complexity using counter method		
1)	6/8/26	Finding complexity - 1		
2)	6/8/26	Finding complexity - 2		
3)	20/8/26	Finding complexity - 3		
4)	20/8/26	Finding complexity - 4		
5)	20/8/26	Finding Complexity - 5		
		II - Divide and Conquer method		
6)	27/8/26	Find the number of zeroes in an array		
7)	27/8/26	majority element		
8)	27/8/26	Finding floor value		
9)	27/8/26	Two element sum to X		
10)	27/8/26	Sorting (Quick sort		
		III - Greedy Algorithms		
11)	29/8/26	1-G - coin problem		
12)	29/8/26	2-G - Cookies problem		
13)	29/8/26	3 - G - Burger problem		
14)	29/8/26	4 - G - Array sum max problem		
15)	29/8/26	5-G - product of array element - Minimum		

Incase of Emergency Please Contact Mobile :

Sl.no	Date	Title	Teacher's Sign
		IV Dinamic programming	
16	2/8/26	1-DP playing with numbers	
17	2/8/26	2-DP playing with chessboard	
18	2/8/26	3-DP longest common subsequence	
19	2/8/26	4-DP longest non-decreasing subsequence	
~~20~~		V - competitive programming	
20	10/10/26	1- finding duplicates O(n^2) Time complexity O(1) space complexity	
21	10/10/26	2 - finding duplicates O(n) Time Complexity O(1) Space complexity	
22	10/10/26	3 - print Intersection of 2 sorted arrays - O(m*n) Time Complexity O(1) space complexity	
23	10/10/26	4 - Intersection of 2 Sorted arrays O(m+n) time complexity O(1) Space Complexity	
24	10/10/26	5 - pair with difference O(n^2) time complexity O(1) space Complexity	
25	10/10/26	6 - pair with difference O(n) Time complexity O(1) space complexity.	

Continued

Ex: 1
6/8/26

Finding Complexity using counter

method - 1

Aim:

To find the counter variable and time Complexity

Algorithm:

```
void function (int n)
{
  int i=0;
  int s=1;
  while (s<=n)
  {
    i++;
    s+=i;
  }
}
```

Program:

```
#include <stdio.h>
int main()
{
 int n, count=0;
 scanf ("%d", &n);
 int s=1; count++;
 int i=1; count++;
 while (s<=n, count++)
 {
    i++ ; count++;
    s+=i ; count++;
 }
 printf ("%d", count);
}
```

Ex: 2

6/8/26

Time complexity:

linear loop, hence O(n)

Sample input	sample output	Got
9	12	12
4	9	9

Result :

thus the time complexity and counter variable is found for the given algorithm.



Ex: 2

6/8/26

Finding complexity using counter method - 2

AIM:

to find the time complexity for a given algorithm using counter method

Algorithm:

```
void func(int n){
  if(n==1)
  {
    printf("*");
  }
  else
  {
    for(int i=1; i<=n; i++)
    {
    for(int j=1; j<=n; j++)
    {
      printf("*");
      printf("*");
      break;
    }
  }
}
```

Program:

```
#include<stdio.h>
int main(){
  int c=0;
  int n;
  scanf("%d", &n);
  if(n==1)
  {
```

```
        c++;
        printf("*");
        c++;
    else{
        c++;
        for(int i=1; c++; i<=n; c++; i++){
        for(int j=1; c++; j<=n; c++; j++){
            c++;
            c++;
            break
        }
    }
}
printf("c");
}
```

Input	output
2	12
1000	5002

Result

The program was written and executed successfully

Ex:3
20/8/26

Finding complexity using counter method :)

Aim:

To find the counter variable and complexity for the given algorithm

Algorithm:

```
for (int i=1; i<=num; ++i)
{
  if (num%i==0)
    {
      printf("%d",i);
    }
}
```

Program:

```
#include <stdio.h>
int main()
{
  int n;
  scanf("%d", &n);
  int c=0;
  int i=0;
  for (i=1; c++, i<=n; ++i, c++)
  {
    if (n%i==0)
      {
        c++;
      }
  }
  printf("%d", c);
}
```

Sample input	Sample output	got
12	31	31
25	54	54

Result:

Thus time complexity and counter variable is found for the given algorithm

EX:4
20/8/24

Finding Complexity using counter method -

Aim:

To find the counter variable and complexity for the given algorithm

Algorithm:

```
void function (int n)
{
 int c=0;
 for (int i=n/2 ; i<n; i++)
    for (int j=1; j<n ; j= 2 * j)
       for (int k=1 ; k<n; k= k *2)
          c++;
}
```

program

```
#include <stdio.h>
int main()
{
   int n;
   scanf ("%d", &n);
   int c=0;
   int count=0;
   for (int i=n/2 ; count++, i<n; i++)
   {
     for (int j=1; count++, j<n; j= 2 * j)
     {
       for (int k=1 ; count++, k<n; k= k *2)
       {
```

```
            ctt;
            count++;
          }
        }
      }
      printf("%d", count);
}
```

Sample Input	Sample output	Got
4	30	30
10	212	212

Result:

Thus time complexity and counter variable is found for the given algorithm.



Ex:5
20/8/26

Finding complexity for an algorithm using counter method.

Aim:

To find the time complexity and counter variable for a given algorithm.

Algorithm:

```
Void reverse (int n)
{
  Int rev = 0 , remainder;
  while (n!=0)
  {
    remainder = n%.10 ;
    rev = rev *10 + remainder ;
    n/=10 ;
  }
  Print (rev);
}
```

Program:

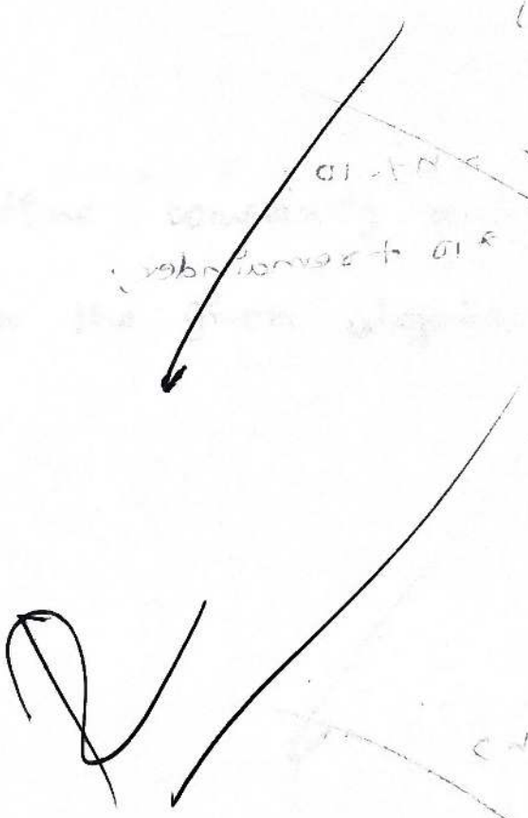
```
#include <stdio.h>
int main()
{
 int n ;
 Scanf ("%d", &n);
 int count=0;
 int rev = 0 , remainder ; count++;
 while (count++, n!=0 )
 {
   remainder = n %.10 ; count ++;
   rev = rev * 10 + remainder ; count ++ ;
   n = n/10 ; count ++;
```

```
    }
    count++;
    printf("%d", count);
}
```

Sample input	Sample output	got
12	11	11

Result:

Thus the time complexity and counter variable a found for the given algorithm.



Ex: 6
26/7/26

II - Divide and Conquer Method

Number of zeros in a Given Array

Aim:

To find the NO. of zeros in a Given array.

Program:

```
# include <stdio.h>
int main()
{
 int n;
 scanf("%d", &n);
 int a[n];
 for(int i=0; i<n; i++)
 {
  scanf("%d", &a[i]);
}
int c=0;
for(int i=0; i<n; i++)
{
if (a[i]==0)
{
 c++;
}
}
}
printf("%d", c);
return 0;
}
```

Input	expected	Got
5 1 1 1 0 0	2	2
10 1 1 1 1 1 1 1 1 1 1	0	0

Output :

The program was written and executed successfully.

Ex:7
27/8/20

Majority Element

Aim:

To write a program to find the majority element.

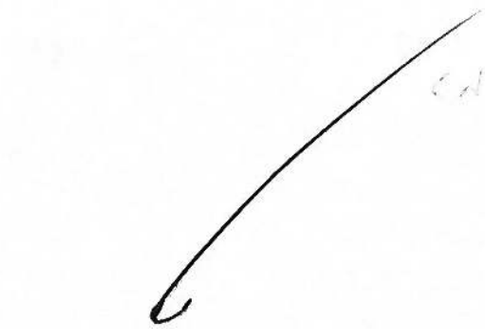
program:

```
#include <stdio.h>
int main()
{
int n;
scanf("%d", &n);
int a[n];
for (int i=0; i<n; i++)
{
scanf("%d", &a[i]);
}
int ct=0;
for (int i=0; i<n; i++)
{
for (int j=0; j<n; j++)
{
if (a[i]==a[j])
{
ct++;
}
}
}
if (ct>n/2)
{
printf("%d", a[i]);
break;
}
}
}
```

Input	Expected	Got
3 3 2 3	3	3

Result:

The program was written and executed successfully.





Ex:8
27/8/26

Finding Floor value

Aim:

To find the floor value of the given array.

Program:

```
#include<stdio.h>
int main()
{
int n;
scanf("%d", &n);
int a[n];
for (int i=0; i<n; i++)
{
  scanf("%d", &a[i]);
}
int key;
scanf("%d", &key);
for (int i=n-1; i>=0; i--)
{
if (a[i]<=key)
{
printf("%d", a[i]);
break;
}
}
}
```

Input	Expected	Got
6 1 2 8 10 12 19 5	2	2

Result

The program was written and executed successfully.



Ex:3
21/8/26

Two Element sum to x

Aim:

To find the two elements sum to x using divide and conquer method

Program:

```
#include <stdio.h>
int main()
{
int n;
scanf("%d", &n);
int a[n];
for(int i=0;i<n;i++)
{
scanf("%d", &a[i]);
}
int key;
int c=0,d=0;
scanf("%d", &key);
for(int i=0; i<n;i++)
{
for(int j=0; j<n; j++)
{
if(a[i]+a[j]==key)
{
 c=a[i];
 d=a[j];
 break;
}
}
}
```

```
int b[2];
if (c<d)
{
b[0]=c;
b[1]=d;
}
else
{
b[0]=d;
b[1]=c;
}
if(c!=0 && d!=0)
{
printf("%d\n%d", b[0], b[1]);
}
else
{
 printf("No");
}
}
```

Input	Expected	Got
4	4	4
2	10	10
4		
8		
10		
14		

Result:

The program was written and executed succesfully.

Ex:10
27/8/26

Implementation of Quick Sort

Aim:

To perform quick sort for sorting the array

Program:

```
#include <stdio.h>
void qsort (int a[], int l, int h)
{
 if (l<h)
 {
  int i=l;
  int j=h;
  int pivot =a[l];
  while (i<j)
  {
   while (a[i] <=pivot && i<h)
   {
    i++;
   }
   while (a[j] > pivot)
   {
    j--;
   }
  if (i<j)
  {
   int temp=a[i];
   a[i] =a[j];
   a[j] =temp;
  }
 }
```

```
        a[i] = a[j];
        a[j] = pivot;
        qsort (a, i, j-1);
        qsort (a, j+1, high);
    }
}
int main()
{
    int n, i;
    int a[100];
    scanf ("%d", &n);
    for (int j=0; j<n; j++)
    {
        scanf ("%d", &a[j]);
    }
    qsort (a, 0, n-1);
    for (i=0; i<n; i++)
    {
        printf ("%d", a[i]);
    }
    return 0;
}
```

Input

5

67 34 12 98 78

expected

12 34 67 78 98

Got

12 34 67 78

Result

Thus the given array is sorted using quick sort.

Ex:11
29/8/26

III - Greedy Algorithms

1-G-Coin problem

Aim:

To write a program to find the minimum number of Indian currency coins / notes required to make a given value v using the available denominations.

program

```
#include <stdio.h>
int main()
{
    int v;
    scanf("%d", &v);
    int d[] = {1000, 500, 100, 50, 20, 10, 5, 2, 1};
    int count=0;
    for (int i=0 ; i<9 ; i++)
    {
        count += v / d[i];
        v %= d[i];
    }
    printf("%d", count);
    return 0;
}
```

34 67 78 98

Input	Expected	Got
49	5	5

Result

The program was written and executed Successfully.

Ex: 12

29/8/26

2-G - Cookies problem

Aim:

To write a program to maximize the number of content children by assigning cookies of sufficient size based on each child's greed factor

Program:

```
#include <stdio.h>
int main()
{
   int n,m;
   scanf("%d",&n);
   int g[n];
   for(int i=0;i<n;i++)
   {
     scanf("%d",&g[i]);
   }
   scanf("%d",&m);
   int s[m],u[m];
   for(int i=0;i<m;i++)
   {
     scanf("%d",&s[i]);
     u[i]=0;
   }
   int count=0;
   for(int i=0;i<n;i++)
   {
     for(int j=0;j<m;j++)
     {
       if(u[j]==0 && s[j]>=g[i])
       {
         count++;
         u[j]=1;
         break;
       }
     }
```

```
    }
    printf("%d", count);
    return 0;
}
```

Ex: 13
29/8/26

Input	Expected	Got
2 1 2 3 1 2 3	2	2

Result:

The program was written and executed Successfully

Ex:13

29/8/26

3-G-Burger problem

Aim:

To write a program to find the minimum distance a person needs to run by sorting the burgers in descending order of calories before eating them.

Program

```
#include <stdio.h>
#include <math.h>
int main()
{
    int n;
    scanf("%d", &n);
    int arr[n];
    for(int i=0; i<n; i++)
    {
      scanf("%d", &arr[i]);
}
int temp;
for(int i=0; i<n; i++)
{
    for(int j=i+1; j<n; j++)
    {
        if(arr[i] < arr[j])
        {
          temp = arr[i];
          arr[i] = arr[j];
          arr[j] = temp;
        }
    }
  }
  int sum = 0;
  for(int i=0; i<n; i++)
  {
     sum += (pow(n,i) * arr[i]);
  }
```

```
    printf("%d", sum);
    return 0;
}
```

Input	Expected	Got
3 1 3 2	18	18

Result

The program was written and executed successfully.

Ex:14
29/8/26

4-G - Array Sum max problem

Aim:

To write the program to maximize the sum of given array

Program:

```
#include <stdio.h>
#include <stdlib.h>
int cmp (const void *a, const void *b)
{
    return *(int*)a - *(int*)b;
}
int main()
{
    int n;
    scanf("%d", &n);
    int a[n];
    for (int i=0; i<n; i++)
    {
        scanf("%d", &a[i]);
    }
    qsort (a, n, sizeof(int), cmp);
    int sum=0;
    for (int i=0; i<n; i++)
    {
        sum+=a[i]*i;
    }
    printf("%d", sum);
}
```

Input	Expected	Got
5	40	40
2		
5		
3		
4		
0		

Result

The program was written and executed successfully.

Ex:15
29/8/24

S-G - Product of array elements - Minimum

Aim:

To write a program to arrange two arrays such that sum of pairwise products is minimized.

program:

```
#include <stdio.h>
#include <stdlib.h>
int compare1(const void *a, const void *b){
    return (*(int*)a - *(int*)b);
}
int compare2(const void *a, const void *b){
    return (*(int*)b - *(int*)a);
}
int main(){
    int n; sum=0;
    scanf("%d", &n);
    int a[n], b[n];
    for(int i=0; i<n; i++)
    for(int i=0; j<n; j++)
    qsort(a, n, sizeof(int), compare1)
    qsort(b, n, sizeof(int), compare2)
    for(int i=0; i<n; i++){
        sum+=(a[i] * b[i]);
    }
    printf("%d", sum);
    return 0;
```

Input	Expected	Got
3 1 2 3 4 5 6	28	28

Result:

Thus the program is executed successfully.

Ex:16
2/8/26

IV - Dynamic programming

1- DP Playing with numbers

Aim

To write a program any efficient to find the possible ways

program

```
#include <stdio.h>
int main(){
    int n;
    if (scanf("%d", &n) !=1) return 0;
    if (n==0 || n==1 || n==2){
       printf("1\n");
       return 0;
    }
    long long dp[n+1];
    dp[0]=1;
    dp[1]=1;
    dp[2]=1;
    for (int i=3; i<=n; i++){
       dp[i]=dp[i-1]+dp[i-3];
    }
    printf("%lld\n", dp[n]);
    return 0;
}
```

Input	Expected	Got
6 25 100	6 8641 2438281959672162 9	6 8641 2438281959672

Result

thus the program was written and executed successfully.



Ex:17
02/8/26

2 - DP playing with chessboard

Aim

To find the maximum monetary value path from the top left cell to the bottom right cell using Dynamic programming moving only right or down

Program

```
#include <stdio.h>
int main(){
    int n;
    scanf("%d", &n);
    int a[n][n];
    int dp[n][n];
    for(int i=0; i<n; i++){
        for(int j=0; j<n; j++){
            scanf("%d", &a[i][j]);
        }
    }
    dp[0][0] = a[0][0];
    for(int j=1; j<n; j++){
        dp[0][j] = dp[0][j-1] + a[0][j];
    }
    for(int i=1; i<n; i++){
        dp[i][0] = dp[i-1][0] + a[i][0];
    }
    for(int i=1; i<n; i++){
        for(int j=1; j<n; j++){
            if(dp[i-1][j] > dp[i][j-1])
                dp[i][j] = dp[i-1][j] + a[i][j];
            else
                dp[i][j] = dp[i][j-1] + a[i][j];
        }
    }
```

```
printf("%d", dp[n-1][n-1]);
return 0;
}
```

Input	Expected	Got
3 1 2 4 2 3 4 8 7 1	19	19
3 1 3 1 1 5 1 4 2 1	12	12

Result

Thus the program was written and executed successfully.

Ex:18
2/8/26

3-DP longest common subsequence

Aim

To find the length of the common longest subsequence between two

program

```
#include <stdio.h>
#include <string.h>
int main() {
    char a[100], b[100];
    int dp[101][101];
    scanf("%s %s", a, b);
    int n = strlen(a), m = strlen(b);
    for (int i=1; i<=n; i++)
        for (int j=1; j<=m; j++)
            if (a[i-1] == b[j-1])
                dp[i][j] = dp[i-1][j-1] + 1;
            else
                dp[i][j] = dp[i-1][j] > dp[i][j-1] ?
                           dp[i-1][j] : dp[i][j-1];
    printf("%d", dp[n][m]);
    return 0;
}
```

Input	Expected	Got
aab	2	2
azb		
ABCD	4	4
ABCD		

Result

thus the program was written and executed Successfully

Ex:29
2/8/26

4-DP longest non-decreasing subsequence

Aim

To find the length of the longest non-decreasing subsequence in a given sequence

program

```
#include<stdio.h>
int main(){
    int n, a[100], dp[100], ans=1;
    scanf("%d", &n);
    for (int i=0; i<n; i++){
        scanf("%d", &a[i]);
        dp[i]=1;
    }
    for (int i=1; i<n; i++)
        for (int j=0; j<i; j++)
            if (a[i]>=a[j] && dp[i]<dp[j]+1)
                dp[i]=dp[j]+1;
    for (int i=0; i<n; i++)
        if (ans<dp[i]) ans=dp[i];

    printf("%d", ans);
    return 0;
}
```

Input	Expected	Got
9 -1 3 4 5 2 2 2 3	6	6
7 1 2 2 4 5 7 6	6	6



Ex:20
10/10/24

V - Competitive programming

1- finding duplicates - O(n^2) Time complexity, O(1) space complexity

Aim

Find the duplicate in array given a read only array of n integers between 1 and n

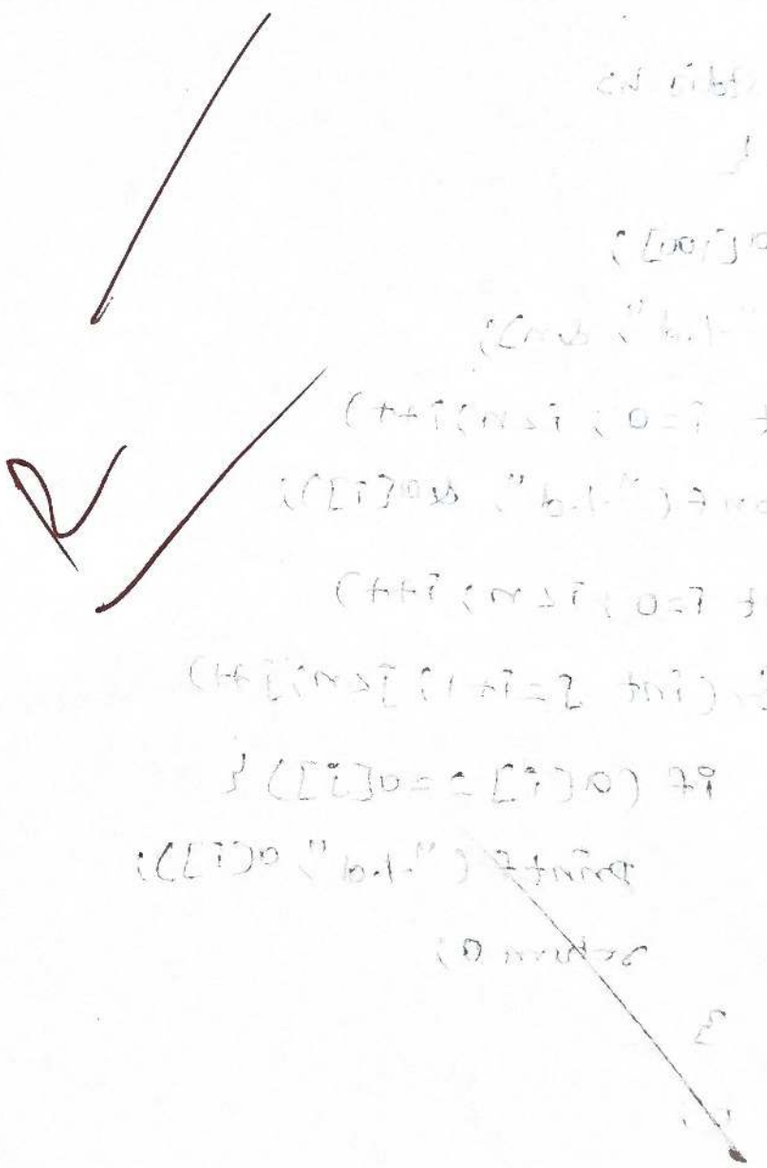
program

```
#include <stdio.h>
int main() {
    int n, a[100];
    scanf("%d", &n);
    for (int i=0; i<n; i++)
        scanf("%d", &a[i]);
    for (int i=0; i<n; i++)
        for (int j=i+1; j<n; j++)
            if (a[i]==a[j]) {
                printf("%d", a[i]);
                return 0;
            }
    return 0;
}
```

Input	Expected	Got
11 10 9 8 7 6 5 4 2 3 8 4 7	7	7

Result

thus the program was written and executed successfully.



Ex:21
10/10/25

2 - Finding Duplicates - O(n) Time Complexity O(1) Space Complexity

Aim

find duplicate in array . And find one number that repeats

Program

```
#include <stdio.h>
int main(){
    int n,a[100];
    scanf("%d", &n);
    for(int i=0; i<n; i++)
        scanf("%d", &a[i]);
    for(int i=0; i<n; i++)
        for(int j=i+1; j<n; j++)
            if(a[i]==a[j]){
                printf("%d", a[i]);
                return 0;
            }
    return 0;
}
```

Input	Expected	Got
11 10 9 8 7 6 5 1 2 3 8 4 7	7	7

Result

Thus the program was written and executed Successfully.

Ex:-22
Id:1026

3- Print Intersection of 2 sorted arrays -
O(m+n) Time Complexity . O(1) Space Complexity

Aim:

Find the Intersection of two sorted arrays and find all the element

program:

```
#include <stdio.h>
int main(){
    int t;
    scanf("%d", &t);
    while(t--){
        int n1, n2;
        scanf("%d", &n1);
        int a[n1];
        for(int i=0; i<n1; i++)
            scanf("%d", &a[i]);
        scanf("%d", &n2);
        int b[n2];
        for(int i=0; i<n2; i++)
            scanf("%d", &b[i]);
        for(int i=0; i<n1; i++){
            for(int j=0; j<n2; j++){
                if(a[i]==b[j]){
                    printf("%d ", a[i]);
                }
            }
        }
    printf("\n");
}
```

return 0;

}

Input	Expected	Got
1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57

Result:

Thus the program was written and executed successfully.



Ex: 23
10/12/26

4- Intersection of 2 sorted arrays - O(m+n) Time Complexity O(1) Space Complexity

Aim

Find the intersection of two sorted arrays and find all the elements which occur in both the arrays.

Program

```
#include <stdio.h>
int main(){
    int t;
    scanf("%d", &t);
    while(t--){
        int m,n;
        scanf("%d", &m);
        int a[m];
        for(int i=0; i<m; i++)
            scanf("%d", &a[i]);
        scanf("%d", &n);
        int b[n];
        for(int i=0; i<n; i++)
            scanf("%d", &b[i]);
        int i=0,j=0;
        while(i<m && j<n){
            if(a[i]==b[j]){
                printf("%d ",a[i]);
                i++;
                j++;
            }
```

```
        else if (a[i] < b[j]) {
            i++;
        }
        else {
            j++;
        }
    }
    printf("\n");
}
return 0;
}
```

Input	Expected	Got
1 3 10 17 51 6 2 7 10 15 51 246	10 51	10 51

Result

Thus the program was written and executed Successfully.

DC:24

10/10/25 5. Pair with difference $O(n^2)$ Time Complexity $O(1)$ Space Complexity

Aim

Give an array of sorted integers and another non-negative integer k.

Program:

```
#include <stdio.h>
int main(){
    int n,k;
    scanf("%d",&n);
    int A[n];
    for(int i=0;i<n;i++)
        scanf("%d",&A[i]);
    scanf("%d",&k);
    for(int i=0;i<n;i++){
        for(int j=i+1;j<n;j++){
            if(A[j]-A[i]==k){
                Printf("1");
                return 0;
            }
        }
    }
    Printf("0");
    return 0;
}
```

Input	Expected	Got
3 1 3 5 4	1	1

Result

Thus the program was written and executed successfully.

Ex:25
10/10/26

6-pair with difference 0(n) Time complexity 0(1) Space complexity

Aim

Give an array of sorted integers and another non-negative integer k

program:

```
#include <stdio.h>
int main() {
    int n, k;
    scanf("%d", &n);
    int A[n];
    for (int i=0; i<n; i++)
        scanf("%d", &A[i]);
    scanf("%d", &k);
int i=0, j=1;
while (i<n && j<n){
  if (i==j) {
      j++;
  }
  else if (A[j]-A[i]==k){
      printf("1");
      return 0;
  }
  else if (A[j]-A[i]<k){
      j++;
  }
  else {
     i++;
  }
}
```

```
    printf("0 ");
    return 0;
}
```

Input	Expected	Got
3 1 3 5 4	1	1